

# Admin-Managed Catalog

What changed on the Kenule branch, how to merge it alongside PR #4 and PR #5, and what FEATURE-005 can now build on.

|            |                                                                                |
|------------|--------------------------------------------------------------------------------|
| AUTHOR     | Mohamed Reda Dahdah (mrdahdah)                                                 |
| FOR        | Kenule (branch owner) · Ghayth (maintainer) · Raead (FEATURE-005) · the team   |
| DATE       | 2 September 2026                                                               |
| REPOSITORY | Ghaythhezami/gmc-ecommerce — base branch feat(kenule)/auth                     |
| THIS WORK  | PR #7 · branch feat/kenule-admin-catalog · 3 commits · 55 files · +2710 / −305 |

### TL;DR

The storefront catalog was hardcoded in a single file, and the admin panel had placeholder pages where Products and Categories should be. PR #7 makes the whole catalog database-driven and editable from the admin panel, splits the Prisma schema into one file per module, and fixes the local launch scripts.

**Before merging, read section 6.** PR #7 re-implements category CRUD that PR #4 already delivers, at the same file paths. The two conflict and must be reconciled — merge #4 first, then rebase #7 onto it.

## 1 Every pull request at a glance

| PR | BRANCH                          | STATUS     | WHAT IT DELIVERS                                                                                                             |
|----|---------------------------------|------------|------------------------------------------------------------------------------------------------------------------------------|
| #7 | feat/kenule-admin-catalog       | OPEN · NEW | Admin-managed catalog: products module, category management UI, dynamic storefront, multi-file Prisma, script fixes.         |
| #6 | fix/production-build-ts59       | OPEN       | Production build fix for TypeScript 5.9 ( <code>ignoreDeprecations</code> ). Effectively a prerequisite for everything else. |
| #5 | feat/kenule-shopping-cart       | OPEN       | FEATURE-004 Shopping Cart — persistent, server-priced, stock-validated.                                                      |
| #4 | feat/kenule-category-management | OPEN       | FEATURE-003 Category Management — admin CRUD API plus public browse, with a unit test.                                       |
| #3 | feat/kenule-review              | OPEN       | Review notes on the auth foundation: Next.js rewrite, RBAC, Cloudinary, guest checkout, multi-file Prisma.                   |
| #2 | feature/shopping-cart           | CLOSED     | Superseded by #5 (retargeted from <code>main</code> onto the Kenule branch).                                                 |
| #1 | feature/category-management     | CLOSED     | Superseded by #4 (retargeted from <code>main</code> onto the Kenule branch).                                                 |

### Suggested merge order

#6 (unblocks the build) → #4 (categories) → #5 (cart) → rebase and merge #7 last, since it touches the most files and overlaps #4.

## 2 What PR #7 changes

### 2.1 Prisma — one file per module

The single `apps/server/prisma/schema.prisma` is replaced by a schema folder, so each domain owns its own file and two students editing different models no longer collide in the same file.

| PATH                                           | CONTAINS                                    |
|------------------------------------------------|---------------------------------------------|
| <code>prisma/schema/schema.prisma</code>       | generator + datasource only                 |
| <code>prisma/schema/user.prisma</code>         | User, Role                                  |
| <code>prisma/schema/category.prisma</code>     | Category                                    |
| <code>prisma/schema/product.prisma</code>      | Product                                     |
| <code>prisma/schema/cart.prisma</code>         | Cart, CartItem                              |
| <code>prisma/schema/order.prisma</code>        | Order, OrderItem, OrderStatus ← FEATURE-005 |
| <code>prisma/schema/notification.prisma</code> | Notification                                |
| <code>prisma/schema/storefront.prisma</code>   | StorefrontAccess                            |

#### Two gotchas that cost real time — please don't re-discover them

**Prisma 6.19 does not auto-detect a schema folder.** The path must be declared explicitly; it now lives in `apps/server/package.json` under the `prisma` key. This prints a deprecation warning pointing at `prisma.config.ts`. That migration was deliberately deferred: a config file disables automatic `.env` loading and would need a `dotenv` dependency.

**Migrations move with the schema.** With a schema *folder*, Prisma resolves migrations relative to that folder, so `prisma/migrations/` is now `prisma/schema/migrations/`. Leave it in the old place and the CLI reports every existing migration as missing and demands a database reset.

### 2.2 New schema fields

One additive migration, `20260902200917_catalog_fields`. No data loss, no reset.

| MODEL    | ADDED                                                                                                                                                                         |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category | <code>description</code> , <code>icon</code> , <code>imageUrl</code> , <code>sortOrder</code> , <code>isActive</code> · index on <code>(isActive, sortOrder)</code>           |
| Product  | <code>compareAtPrice</code> , <code>rating</code> , <code>reviewCount</code> , <code>isFeatured</code> , <code>isActive</code> · index on <code>(isActive, isFeatured)</code> |

`compareAtPrice` is the “was” price. The discount percentage is *derived server-side*, never stored, so it cannot drift out of sync with the price.

## 2.3 API surface

| VERB                  | ROUTE                      | ACCESS | NOTES                                                                                                                                                  |
|-----------------------|----------------------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET                   | /api/categories            | Public | Active categories, ordered by <code>sortOrder</code> , with product counts                                                                             |
| GET                   | /api/categories/:slug      | Public | One category                                                                                                                                           |
| GET                   | /api/categories/manage/all | Admin  | Includes hidden categories                                                                                                                             |
| POST / PATCH / DELETE | /api/categories            | Admin  | Delete refuses while products remain                                                                                                                   |
| GET                   | /api/products              | Public | <code>category</code> , <code>search</code> , <code>featured</code> , <code>onSale</code> , <code>sort</code> , <code>page</code> , <code>limit</code> |
| GET                   | /api/products/:slug        | Public | One product                                                                                                                                            |
| GET                   | /api/products/manage/all   | Admin  | Includes hidden products                                                                                                                               |
| POST / PATCH / DELETE | /api/products              | Admin  | Delete refuses if the product is on an order                                                                                                           |

Admin routes sit behind `JwtAuthGuard` + `RolesGuard` + `@Roles(Role.ADMIN)`, matching the pattern Kenule established in `auth/`. The literal `manage/all` route is declared *before* `:slug` so the parameterised route does not swallow it.

## 2.4 Three implementation details worth knowing

- **Decimal → number.** Prisma returns `Decimal` instances that JSON-serialize as *strings*, which silently breaks arithmetic in the browser. Both services map them to numbers before returning. Any new module reading `price` should do the same.
- **Column-to-column filtering.** `onSale` compares `compareAtPrice > price` using Prisma *field references* inside `where`. Filtering in JS after the query would corrupt `total` and produce short pages.
- **Deactivate, don't delete.** Products referenced by an `OrderItem` cannot be deleted; the API returns a message telling the admin to deactivate instead. This protects order history — directly relevant to FEATURE-005.

## 2.5 Admin panel and storefront

- Admin gains real **Categories** and **Products** pages (full CRUD, category filter, modal forms) built on the tokens already in `styles.css`. They replace the placeholder pages.
- `AdminLayout` used Tailwind classes that **do not exist** in the admin theme — `bg-bg`, `bg-surface`, `border-border`, `text-text`, `text-star` — while `styles.css` defines `--color-admin-*`. They compiled to nothing, leaving the background, body text, panel surfaces and header border unstyled. Now mapped onto the tokens that exist.
- `apps/client/src/components/home/data.ts` is **deleted**. Category nav, category grid, hero rails, flash sale and trending rows all read from the API.
- The catalog page gains search, sort and pagination and serves both `/products` and `/category/:slug`; `/products/:slug` is now a real product detail page.
- Cross-app navigation was missing entirely: the admin sidebar gains *View storefront*, and the storefront top bar gains *Home* and *Admin*. Separate origins, so these are plain anchors driven by `VITE_STOREFRONT_URL` / `VITE_ADMIN_URL`.

### 3 Running it locally

Two helper scripts sit at the repository root. Both were broken in ways that are easy to misread as “the project doesn't work”, and both are fixed in PR #7.

| SCRIPT            | WHAT IT DOES                                                                                                                                                                                                                                                                     |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| launch-webapp.bat | Starts the <code>gmc-postgres</code> container on port <b>5433</b> , waits for it, runs <code>prisma generate</code> and <code>migrate deploy</code> , compiles the server once, then opens three terminals (API, storefront, admin) and polls the API before reporting success. |
| stop-webapp.bat   | Frees ports 3000 / 5173 / 5174, closes the three terminals, stops the container. Data survives in the Docker volume.                                                                                                                                                             |

#### Why the API used to die on launch while the two frontends came up fine

The script deleted `apps\server\dist` and immediately ran `nest start --watch`. The watcher launches `dist/src/main.js` the moment it appears, so against an empty dist it *raced the compiler* and exited with `Cannot find module './app.module'`. Nest only retries on a file change, so the API stayed dead and silent. It now compiles once before the watcher starts.

Two smaller fixes: `timeout /t 1` aborts with “input redirection is not supported” when stdin is redirected, so the DB wait loop uses `ping`; and the script now warns explicitly instead of printing the URLs while the API is down.

#### If the API will not start

Check `docker ps` first. A stopped `gmc-postgres` makes Nest exit at boot with Prisma `P1001 - Can't reach database server at localhost:5433`. Note the port is **5433**, not the default 5432.

**Seed accounts** all use `Student123! : admin@example.com` (ADMIN), `student1@example.com`, `student2@example.com`. Note the storefront login at `/api/auth/login` deliberately rejects ADMIN accounts; admins sign in at `/api/admin/auth/login`.

### 4 Catalog data

The seed now creates a coherent catalog — 6 categories and 12 products with real prices, stock, ratings, sale prices and imagery — and is idempotent on slug, so re-running it is safe.

#### Data cleanup applied to the shared dev database

The original starter seed had left `Electronics` and `Home` categories holding a *Wireless Headphones* and a *Desk Lamp* — placeholders that do not belong in a games catalog and were appearing in the storefront navigation. Both products were reassigned to `Accessories` and the two empty categories were removed. Current state: **6 categories, 14 products, 0 products with a dangling category**.

### 5 FEATURE-003 and FEATURE-004 — verification

Both were reviewed against their tickets in `docs/tickets/`. Both are implemented and meet their acceptance criteria.

| ACCEPTANCE CRITERION                             | MET | EVIDENCE                                                                                       |
|--------------------------------------------------|-----|------------------------------------------------------------------------------------------------|
| <b>FEATURE-003 — Category Management (PR #4)</b> |     |                                                                                                |
| Only admins mutate                               | Yes | Guards on every write; verified 401 unauthenticated, 403 with a CUSTOMER token                 |
| Unique names / slugs                             | Yes | Pre-write check returning a clean 409 instead of a raw Prisma P2002                            |
| DTO validation                                   | Yes | <code>class-validator</code> DTOs behind the global ValidationPipe                             |
| RTK Query invalidation                           | Yes | Tag-based invalidation on the client slice                                                     |
| Admin + public UI paths                          | Yes | Public categories page; admin CRUD arrives with PR #7                                          |
| <b>FEATURE-004 — Shopping Cart (PR #5)</b>       |     |                                                                                                |
| Add / remove / update / clear                    | Yes | All four implemented in <code>cart.service.ts</code>                                           |
| Quantities validated                             | Yes | <code>assertStock</code> rejects quantities above available stock                              |
| Totals use server prices                         | Yes | Line totals and subtotal computed from <code>product.price</code> , never from the client      |
| Users only access their cart                     | Yes | Every query scoped by <code>userId</code> ; removal uses a cart-scoped <code>deleteMany</code> |
| Cart UI states                                   | Yes | <code>CartPage.tsx</code> handles empty / loading / error                                      |

### Neither is merged yet — and the cart is not usable on the Kenule branch alone

On `feat(kenule)/auth` there is no `/cart` route, the cart badge is hardcoded to `0`, and “Add to Cart” does nothing. Kenule's client-only `features/cart/cartSlice.ts` and PR #5's server-backed cart are two competing sources of truth — **the team needs to pick one**. Prefer the server cart: it is the only one that can price an order correctly.

## 6 The conflict between PR #7 and PR #4 — please read

PR #4 already delivers FEATURE-003. PR #7 was built before that was taken into account and **independently adds its own `apps/server/src/categories/` module at the same file paths**. Both cannot be merged as they stand.

|                                                                                            | PR #4                                   | PR #7                                |
|--------------------------------------------------------------------------------------------|-----------------------------------------|--------------------------------------|
| Category CRUD API                                                                          | Yes                                     | Yes                                  |
| Unit test ( <code>categories.service.spec.ts</code> )                                      | Yes                                     | No                                   |
| Products-by-category                                                                       | <code>/categories/:slug/products</code> | <code>/products?category=slug</code> |
| Uniqueness conflict status                                                                 | 409                                     | 400                                  |
| <code>icon</code> / <code>sortOrder</code> / <code>isActive</code> / <code>imageUrl</code> | No                                      | Yes                                  |
| Admin CRUD UI                                                                              | No                                      | Yes                                  |
| Products module                                                                            | No                                      | Yes                                  |
| Storefront wired to the API                                                                | categories page only                    | <b>whole storefront</b>              |

### Recommended resolution

Merge **#4** first and keep its service and its test as the canonical category implementation. Then rebase **#7** onto the result and re-apply only three things: the extra `Category` columns, the products module, and the admin UI. Everything in #7 outside `categories/` — products, the storefront rewrite, the Prisma split, the script fixes — is additive and should rebase cleanly.

## 7 For Raeed — what FEATURE-005 can build on

Orders is a module plus a frontend slice. `Order`, `OrderItem` and `OrderStatus` already exist in Prisma, so **no migration is required**. Once PR #7 lands, these are ready to use:

- **A products module to read from.** `ProductsService` already returns prices as numbers, so order totals can be computed without unwrapping `Decimal` yourself.
- **Stock and price on every product.** `stock` for availability checks, `price` for the `unitPrice` snapshot on each `OrderItem`.
- **Deletion already protects order history.** A product on an order line cannot be deleted, so `OrderItem.productId` will not dangle. Keep this rule in mind if you add order-side deletes.
- **An admin shell to extend.** Add an Orders page next to Categories and Products; reuse the shared primitives in `apps/admin/src/components/ui.tsx` ( `Field`, `Input`, `Select`, `Modal`, `Banner` ) and the `errorMessage` helper so validation errors surface properly. The sidebar already has an `/orders` placeholder route to replace.
- **A guard pattern to copy.** `products.controller.ts` is the shortest example of public reads plus admin-only writes on one controller, including the `manage/all` -before- `:slug` route-ordering rule.

### Your one real decision: which cart feeds an order

An order is created from a cart, and there are currently two carts. Design against PR #5's **server cart** — it computes subtotals from database prices, which is what an order needs. If #5 has not merged when you start, branch from `feat/kenule-shopping-cart` rather than from the Kenule branch, or build the Orders module against the Prisma models directly and wire the cart link once #5 lands.

### Snapshot the price at purchase time

`OrderItem.unitPrice` exists precisely so an order keeps the price that was charged. Copy `product.price` into it when the order is created — never join back to `Product` to display a historical order, or every past order changes whenever an admin edits a price in the panel.

## 8 Commits in PR #7

| COMMIT  | SUBJECT                                                         |
|---------|-----------------------------------------------------------------|
| 8c76844 | feat(catalog): admin-managed products and categories            |
| 7fc0d8c | fix(ui): repair admin theme tokens and add cross-app navigation |
| 4b474cb | chore(scripts): fix the local launch/stop helpers               |

### Verification performed before opening the PR

- `tsc --noEmit` clean on server, admin and client (with PR #6 applied — without it the build fails on TypeScript 5.9).
- Guards: 401 unauthenticated, 403 with a CUSTOMER token on an admin write.
- Validation: 400 for `compareAtPrice ≤ price`; 400 deleting a category that still holds products.
- Full round trip through the admin UI — created a product, confirmed slug generation and the derived discount, saw it appear on the storefront category page and its detail page, then deleted it and confirmed a 404.
- Both scripts run end to end: `stop` frees all three ports and stops the container; `launch` brings all three services up.